Noida Institute of Engineering & Technology, Greater Noida

NIET
Greater Noida
Autonomous Institute
Get Future Ready

LAB FILE

Branch : EE (VLSI)

Semester : 3rd semester

Session : 2025 - 2026

Subject Code : BEC0356

Faculty Name : Mr. Prem Kumar

Name & Roll No : Aditya Raj (0241ECVLS013)

Subject Name : Data Structure Lab

CodeTantra

NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY
19, KNOWLEDGE PARK-II, INSTITUTIONAL AREA,
GREATER NOIDA, (U. P.) - 201 306, INDIA

NIET Greater Noida	PROGRAM LIST	Session:2025-26
		Semester:III

S.No	Program Name	Submission Date	Grade	Signature	Remark
1	Maximum Element Of Array				
2	Sum of All Elements in an Array				
3	Reverse the Elements of an Array				
4	Check If Array Is Sorted in Ascending Order				
5	Count Occurrence of an Element in an Array				
6	Creation And Traversal of 2D Array				
7	Transpose of a Given Matrix				
8	Check If Matrix Is Sparse and Print Sparse Representation				
9	Triplex Representation				
10	Linear search Technique				
11	Binary Search				
12	Selection Sort Algorithm				
13	Bubble Sort Algorithm				
14	Insertion Sort Algorithm				
15	Shell Sort Algorithm				
16	Counting Sort Algorithm				
17	Singly Linked List				
18	Double - Linked List Implementation				
19	Operations on Circular Linked List				

NIET Greater Noida	PROGRAM LIST	Session:
		Semester:

S.No	Program Name	Submission Date	Grade	Signature	Remark
20	Doubly Circular Linked List				
21	Reverse the Linked List				
22	Palindrome Check for a Singly Linked List				
23	Reverse of a Double Linked List				
24	Midpoint of Linked List				
25	Midpoint of a double linked list				
26	Merging two sorted linked lists				
27	Detect Loop in a Linked List				
28	Polynomial Addition				
29	Stack Implementation using Arrays				
30	Stack Implementation using Linked Lists				
31	Convert Infix Expression to Equivalent Postfix Expression.				
32	Convert Infix Expression to Equivalent Postfix Expression.				
33	Reverse a Stack using Recursion				
34	Binary Search				

NIET Greater Noida	PROGRAM LIST	Session:
		Semester:

S.No	Program Name	Submission Date	Grade	Signature	Remark
35	Fibonacci Numbers				
36	Tower of Hanoi Puzzle				
37	Queue Implementation using Arrays				
38	Circular Queue using Arrays				
39	Queue Implementation using Stack				
40	Write a program to implement the Priority Queue using Heaps.				
41	Implementation of Double Ended Queue using Linked List				
42	Merge sort of a linked list				
43	Quick Sort (Divide and Conquer) using recursion				
44	Merge Sorted Array				
45	Quick Sort(Divide and Conquer)				
46	Fractional Knapsack Problem				
47	Activity Selection Problem				
48	Job assignment problem				

Aim:

Implement a program to find the maximum element in an array of size n.

Input Format:

- The first line of the input represents an integer n
- The second line of the input represents n space separated elements of the array (Integers)

Output Format:

- The first line of the output represents the input array.
- The second line contains an integer representing the maximum element of the array.

Program:

CTC37586.c

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("[");
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) {
            printf(", ");
        }
    }
    printf("]\n");

    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    printf("%d\n", max);

    return 0;
}
```

Output:

Test case - 1
User Output
4
7 2 9 5
[7, 2, 9, 5]

9

Test case - 2
User Output
3
-1 -4 0
[-1, -4, 0]
0

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program to find the sum of all elements in an array of size n.

Input Format:

- The first line of the input represents an integer n
- The second line of the input represents n space separated elements of the array (Integers)

Output Format:

- The first line of the output represents the input array.
- The second line of the output contains an integer representing the sum of all element of the array.

Program:

CTC37596.c

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    int sum = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
    }

    printf("[");
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) {
            printf(", ");
        }
    }
    printf("]\n");

    printf("%d\n", sum);

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
3
1 2 3
[1, 2, 3]
6

Test case - 2
User Output

5
4 -2 7 1 0
[4, -2, 7, 1, 0]
10

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program to reverse the elements of an array of size n.

Input Format:

- The first line of the input contains an integer n representing the number of elements.
- The second line of the input represents n space separated elements of the array (Integers).

Output Format:

- The output represents the elements of the input array in reversed order.

Program:

CTC37597.c

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("[");
    for (int i = n - 1; i >= 0; i--) {
        printf("%d", arr[i]);
        if (i != 0) printf(", ");
    }
    printf("]\n");

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
3
1 2 3
[3, 2, 1]

Test case - 2
User Output
6
0 -1 2 -3 4 -5
[-5, 4, -3, 2, -1, 0]

Result:

Thus the above program is executed successfully and the output has been verified

Aim:

Implement a program to check if an array of size n is sorted in ascending order or not.

Input Format:

- The first line of the input represents an integer n
- The second line of the input represents n space separated elements of the array (Integers)

Output Format:

- The output represents **True** if the array is sorted in ascending order, otherwise, it is **False**

Program:

CTC37601.c

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int is_sorted = 1;

    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            is_sorted = 0;
            break;
        }
    }

    if (is_sorted)
        printf("True\n");
    else
        printf("False\n");

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
5
1 2 3 4 5
True

Test case - 2
User Output
4

```
False
```

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program to count the occurrence of a specific element k in an array of size n

Input Format:

- The first line of the input represents an integer n
- The second line of the input represents n space separated elements of the array (Integers)
- The third line of the input represents an integer k to search the occurrence in the array.

Output Format:

- The output contains an integer representing the count of the occurrence of integer k in the array.

Program:

CTC37615.c

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            count++;
        }
    }

    printf("%d\n", count);

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
5
1 2 3 2 4
2
2

Test case - 2
User Output
4

6
0

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program that takes the number of rows r and columns c from the user to create a 2D array, and then traverses and prints the array in both row-major and column-major order.

Input Format:

- The first line of input contains two integers r and c separated by a space, representing the number of rows and columns, respectively.
- The second line of input contains $r \times c$ space-separated elements to populate the 2D array.

Output Format:

- The first r lines of the output should each contain c space-separated elements, representing the row-major order traversal of the array.
- The next c lines of the output should each contain r space-separated elements, representing the column-major order traversal of the array.

Program:

CTC37640.c

```
#include <stdio.h>

int main() {
    int rows, cols;
    scanf("%d %d", &rows, &cols);

    int arr[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &arr[i][j]);
        }
    }

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
2 3

1 2 3 4 5 6
1 2 3
4 5 6
1 4
2 5
3 6

Test case - 2
User Output
2 2
1 3 2 4
1 3
2 4
1 2
3 4

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program that takes the number of rows r and columns c from the user to create a matrix and prints both the original matrix and its transpose.

Input Format:

- The first line of input contains two integers r and c separated by a space, representing the number of rows and columns, respectively.
- The next r lines each contain c space-separated elements to populate the matrix.

Output Format:

- The first r lines of the output should each contain c space-separated elements, representing the original matrix.
- The next c lines of the output should each contain r space-separated elements, representing the transpose of the matrix.

Program:

CTC37642.c

```c
#include <stdio.h>

int main() {
    int rows, cols;
    scanf("%d %d", &rows, &cols);

    int matrix[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

Test case - 1
User Output
2 3

1 2 3
4 5 6
1 2 3
4 5 6
1 4
2 5
3 6

Test case - 2
User Output
3 2
7 8
9 10
11 12
7 8
9 10
11 12
7 9 11
8 10 12

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Implement a program that determines whether a given matrix is sparse and, if so, prints its non-zero elements. A matrix is considered sparse if the number of zero elements is greater than the number of non-zero elements.

Input Format:

- The first line of input contains two integers ***r*** and ***C*** separated by a space, representing the number of rows and columns, respectively.
- The nextr lines each contain ***C*** space-separated integers, representing the elements of the matrix.

Output Format:

- The first line of the output should be a boolean value (True or False), indicating whether the matrix is sparse.
- If the matrix is sparse, print the non-zero elements in the format <row> <column> <value>, each on a new line. If the matrix is not sparse, print nothing.

Program:

CTC37649.c

```c
#include <stdio.h>

int main() {
    int rows, cols;
    scanf("%d %d", &rows, &cols);

    int matrix[rows][cols];
    int zero_count = 0, non_zero_count = 0;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
            if (matrix[i][j] == 0)
                zero_count++;
            else
                non_zero_count++;
        }
    }

    if (zero_count > non_zero_count) {
        printf("True\n");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (matrix[i][j] != 0) {
                    printf("%d %d %d\n", i, j, matrix[i][j]);
                }
            }
        }
    } else {
         printf("False\n");
    }

    return 0;
}
```

Output:

Test case - 1
User Output
3 3
0 0 3
0 0 6
0 0 0
True
0 2 3
1 2 6

Test case - 2
User Output
3 3
1 2 3
4 0 6
7 8 9
False

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Imagine you are working on a project for a data analytics company. They have a dataset that consists of a large matrix where each cell represents a measurement taken at different times and locations. This matrix is quite large, and most of its elements are zeros, which means it is a sparse matrix

Your task is to develop a tool that will help the company analyze this matrix efficiently.

Matrix Input:

- Create a tool that allows the user to input the dimensions of the matrix (number of rows and columns) and then enter the elements of the matrix. Each element is an integer.

Matrix Analysis:

- Implement functionality to determine if the matrix is sparse. A matrix is considered sparse if more than half of its elements are zeros.

Sparse Matrix Representation:

- If the matrix is identified as sparse, convert it to its triplet form representation. The triplet form should consist of a list of tuples where each tuple represents a non-zero element in the format (row_index, column_index, value).

Output Requirements:

- If the matrix is sparse, print its triplet representation.

Input Format

- First Line: Two integers r and c representing the number of rows and columns of the matrix separated by a space
- Following Lines: r rows represents c elements with a new line for every i^{th} row where the elements are separated by a space

Output Format

- Triplet Representation (if sparse): Print the triplet representation of the matrix. Each triplet should be printed on a new line in the format (row_index, column_index, value).
- if not sparse : **Return -1**

Constraints

- 1 <= rows, cols <= 1000
- -10^6 <= matrix[i][j] <= 10^6 (matrix elements range between -1,000,000 and 1,000,000)
- The input matrix will be a rectangular grid (i.e., all rows have the same number of columns).

Program:

CTC38898.c

CodeTantra

```c
#include <stdio.h>

int main() {
    int rows, cols;
    scanf("%d %d", &rows, &cols);

    int matrix[rows][cols];
    int zero_count = 0;
    int non_zero_count = 0;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
            if (matrix[i][j] == 0)
                zero_count++;
            else
                non_zero_count++;
        }
    }

    int total_elements = rows * cols;

    if (zero_count > total_elements / 2 && non_zero_count > 0) {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (matrix[i][j] != 0) {
                    printf("%d %d %d\n", i, j, matrix[i][j]);
                }
            }
        }
    } else {
        printf("-1\n");
    }

    return 0;
}
```

Output:

Test case - 1
User Output
2 2
0 0
0 0
-1

Test case - 2
User Output
3 3
0 1 0

CodeTantra

0 4 0
0 1 1
1 0 2
1 2 3
2 1 4

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program to check whether a given element is present in an array using linear search, if element is found print the index of its first occurrence if the element is not found print "Not found".

Input format:

- The first line of input contains an integer representing the no. of elements of the array.
- The second line of input contains the array of integers separated by space.
- The last line of input contains the key element to be searched.

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Sample Test Case:

Input:

7
1 2 3 4 3 5 6
3

Output:

2

Program:

CTC17129.c

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int key;
    scanf("%d", &key);

    int found = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            printf("%d\n", i);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Not found\n");
    }

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
7
1 2 3 4 3 5 6
3
2

Test case - 2
User Output
10
1 2 3 4 5 6 7 8 9 19
20
Not found

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given a list of N numbers $a_1 + a_2 + a_3 + \ldots\ldots + a_n$. Find The Position of number x in the given list using binary search.

Input Format:

- The first line takes the input value of N.
- The second line takes input N space-separated integer values.
- The third line takes the input value of x which needs to be searched.

Output Format:

- Output is an integer that represents the position of x in the given list. Otherwise print -1.

Constraints:

- $0 < N < 1000$
- $0 <$ element of array < 100
- $0 < x < 100$

Note:

- The default position of first element is 0, and find the element's position in the sorted order of occurrence. Print -1 if not found.
- For repeated elements find the position of the last occurrence.

Program:

CTC14028.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int binary_search_last_occurrence(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            result = mid;
            low = mid + 1;
        } else if (arr[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return result;
}

int main() {
    int n, x;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);
    qsort(arr, n, sizeof(int), compare);
    int pos = binary_search_last_occurrence(arr, n, x);
    printf("%d\n", pos);

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
5
1 5 8 7 4
8
4

Test case - 2
User Output
5
12 85 97 45 2
145
-1

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given an array of integers. Your task is to sort the array in ascending order using the **Selection Sort algorithm**.

Input Format:

- The first line contains a single integer n, which represents the number of elements in the array.
- The second line contains n space-separated integers, which are the elements of the array.

Output Format:

- Print the sorted array on a single line, with elements separated by spaces.
- If the number of elements does not match n, print -1 and do not perform any sorting.

Constraints:

- The number of elements n is between 1 and 10^3.
- Each integer in the array is between -10^6 and 10^6.

Program:

CTC38968.c

CodeTantra

```c
#include <stdio.h>
#include <string.h>
#include  <stdlib.h>

void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        if  (min_idx  !=  i)  {
            int temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx]  =  temp;
        }
    }
}

int main() {
    int n;
    if (scanf("%d\n", &n) != 1) {
        printf("-1\n");
        return 0;
    }

    char line[5000];
    if (!fgets(line, sizeof(line), stdin)) {
        printf("-1\n");
        return 0;
    }

    int arr[1000];
    int count = 0;
    char *token = strtok(line, " \n");
    while (token != NULL) {
        if (count >= n) {
            printf("-1\n");
            return 0;
        }
        arr[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    if (count != n) {
        printf("-1\n");
        return 0;
    }

    selection_sort(arr,  n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");
```

CodeTantra

```
    return 0;
}
```

Output:

Test case - 1
User Output
6
10 5 3 8 6 7
3 5 6 7 8 10

Test case - 2
User Output
3
2 4 6 8
-1

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a task to sort an array of integers using the Bubble Sort algorithm. Your program should follow these specifications:

Input Format:

- The first line contains a single integer **n**, which represents the number of elements in the array.
- The second line contains**n** space-separated integers, which are the elements of the array.

Output Format:

- Print the sorted array on a single line, with elements separated by spaces.
- If the number of elements does not match**n**, print **-1** and do not perform any sorting.

Constraints:

- $1 \leq n < 10^3$
- $10^{-6} \leq$ Each integer in the array $\leq 10^6$

Program:

CTC38978.c

CodeTantra

```c
#include <stdio.h>
#include <string.h>
#include  <stdlib.h>

void bubble_sort(int arr[], int n) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    if (scanf("%d\n", &n) != 1) {
        printf("-1\n");
        return 0;
    }

    char line[5000];
    if (!fgets(line, sizeof(line), stdin)) {
        printf("-1\n");
        return 0;
    }

    int arr[1000];
    int count = 0;
    char *token = strtok(line, " \n");
    while (token != NULL) {
        if (count >= n) {
            printf("-1\n");
            return 0;
        }
        arr[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    if (count != n) {
        printf("-1\n");
        return 0;
    }

    bubble_sort(arr,  n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");

    return 0;
```

CodeTantra

```
}
```

Output:

Test case - 1
User Output
6
10 5 3 8 6 7
3 5 6 7 8 10

Test case - 2
User Output
3
2 4 6 8
-1

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a task to sort an array of integers using the Insertion Sort algorithm. Your program should follow these specifications:

Input Format:

- The first line contains a single integer **n**, which represents the number of elements in the array.
- The second line contains**n** space-separated integers, which are the elements of the array.

Output Format:

- Print the sorted array on a single line, with elements separated by spaces.
- If the number of elements does not match**n**, print **-1** and do not perform any sorting.

Constraints:

- $1 \leq n < 10^3$
- $10^{-6} \leq$ Each integer in the array $\leq 10^6$

Program:

CTC38981.c

CodeTantra

```c
#include <stdio.h>
#include <string.h>
#include  <stdlib.h>

void insertion_sort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

int main() {
    int n;
    if (scanf("%d\n", &n) != 1) {
        printf("-1\n");
        return 0;
    }

    char line[5000];
    if (!fgets(line, sizeof(line), stdin)) {
        printf("-1\n");
        return 0;
    }

    int arr[1000];
    int count = 0;
    char *token = strtok(line, " \n");
    while (token != NULL) {
        if (count >= n) {
            printf("-1\n");
            return 0;
        }
        arr[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    if (count != n) {
        printf("-1\n");
        return 0;
    }

    insertion_sort(arr,  n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");

    return 0;
```

CodeTantra

```
}
```

Output:

Test case - 1
User Output
6
10 5 3 8 6 7
3 5 6 7 8 10

Test case - 2
User Output
3
2 4 6 8
-1

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a task to sort an array of integers using the Shell Sort algorithm. Your program should follow these specifications:

Input Format:

- The first line contains a single integer **n**, which represents the number of elements in the array.
- The second line contains**n** space-separated integers, which are the elements of the array.

Output Format:

- Print the sorted array on a single line, with elements separated by spaces.
- If the number of elements does not match**n**, print **-1** and do not perform any sorting.

Constraints:

- The number of elementsn is between 1 and 1000.
- Each integer in the array is between -1,000,000 and 1,000,000.

Program:

CTC38982.c

CodeTantra

```c
#include <stdio.h>
#include <string.h>
#include  <stdlib.h>

void shell_sort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}

int main() {
    int n;
    if (scanf("%d\n", &n) != 1) {
        printf("-1\n");
        return 0;
    }

    if (n < 1 || n > 1000) {
        printf("-1\n");
        return 0;
    }

    char line[5000];
    if (!fgets(line, sizeof(line), stdin)) {
        printf("-1\n");
        return 0;
    }

    int arr[1000];
    int count = 0;
    char *token = strtok(line, " \n");
    while (token != NULL) {
        if (count >= n) {
            printf("-1\n");
            return 0;
        }
        arr[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    if (count != n) {
        printf("-1\n");
        return 0;
    }

    shell_sort(arr,  n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(" ");
```

CodeTantra

```
    }
    printf("\n");

    return 0;
}
```

Output:

Test case - 1
User Output
6
10 5 3 8 6 7
3 5 6 7 8 10

Test case - 2
User Output
3
2 4 6 8
-1

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a task to sort an array of integers using the Counting Sort algorithm. Your program should follow these specifications:

Input Format:

- The first line contains a single integer **n**, which represents the number of elements in the array.
- The second line contains**n** space-separated integers, which are the elements of the array.

Output Format:

- Print the sorted array on a single line, with elements separated by spaces.
- If the number of elements does not match**n**, print **-1** and do not perform any sorting.

Constraints:

- The number of elementsn is between 1 and 1000.
- Each integer in the array is between -1,000,000 and 1,000,000.

Program:

CTC38983.c

CodeTantra

```c
#include <stdio.h>
#include <string.h>
#include  <stdlib.h>

#define OFFSET 1000000  // To shift negative numbers to positive indices
#define  MAX_RANGE  2000001   //  Range  from  -1,000,000  to  1,000,000  inclusive

int main() {
    int n;
    if (scanf("%d\n", &n) != 1) {
        printf("-1\n");
        return 0;
    }

    if (n < 1 || n > 1000) {
        printf("-1\n");
        return 0;
    }

    char line[5000];
    if (!fgets(line, sizeof(line), stdin)) {
        printf("-1\n");
        return  0;
    }

    int arr[1000];
    int count = 0;
    char *token = strtok(line, " \n");
    while (token != NULL) {
        if (count >= n) {
            printf("-1\n");
            return 0;
        }
        arr[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    if (count != n) {
        printf("-1\n");
        return 0;
    }

    int  freq[MAX_RANGE]  =  {0};

    for (int i = 0; i < n; i++) {
        freq[arr[i] + OFFSET]++;
    }

    for (int i = 0; i < MAX_RANGE; i++) {
        while (freq[i] > 0) {
            printf("%d", i - OFFSET);
            freq[i]--;
            if (n > 1) {
                printf("  ");
            }
            n--;
        }
```

CodeTantra

```
    }
    printf("\n");

    return 0;
}
```

Output:

Test case - 1
User Output
6
10 5 3 8 6 7
3 5 6 7 8 10

Test case - 2
User Output
3
2 4 6 8
-1

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that uses functions to perform the following **operations on a singly linked list.**

i) Insertion ii) Deletion

Input Format:

- The first line of input contains the number of operations to be performed on the singly linked list
- The next lines contain the integers separated by a space, in which the first integer indicates the operation to be performed and the second integer contains the element to be inserted or deleted.
- **1 --->** indicates the insertion
- **2 --->** indicates the deletion

Output Format:

- Display the elements of the single-linked list after performing all the operations.

Sample Test Case:

Input:

```
4
1 4
1 5
1 6
2 5
```

Output:

```
4 6
```

Explanation:

1 4 ---> 4 will be inserted
1 5 ---> 5 will be inserted
1 6 ---> 6 will be inserted
2 5 ---> 5 will be deleted
4 Operations are completed, The final output is the linked list ---> **4 6**

Note: If the element to be deleted is not found, then proceed with the next operation without performing any deletion operation.

Program:

CTC17087.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

void insert(Node** head_ref, int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    Node* temp = *head_ref;
    while (temp->next != NULL) temp = temp->next;
    temp->next = new_node;
}

void delete(Node** head_ref, int key) {
    Node* temp = *head_ref;
    Node* prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}

void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int operations;
    if (scanf("%d", &operations) != 1) return 0;

    Node* head = NULL;
    for (int i = 0; i < operations; i++) {
        int op, val;
        if (scanf("%d %d", &op, &val) != 2) return 0;
```

CodeTantra

```
        if (op == 1) insert(&head, val);
        else if (op == 2) delete(&head, val);
    }

    printList(head);

    while (head != NULL) {
        Node* temp = head;
        head = head->next;
        free(temp);
    }
    return 0;
}
```

Output:

Test case - 1
User Output
4
1 4
1 5
1 6
2 5
4 6

Test case - 2
User Output
7
1 22
1 33
2 44
1 55
1 66
2 55
1 77
22 33 66 77

CodeTantra

Result:
Thus the above program is executed successfully and the output has been verified

Aim:

Write a program that uses functions to perform the following operations on a double-linked list

The operations performed on the **double-linked list** are

1. Insertion
2. Deletion
3. Traversal

Input Format:

- The first line of input contains the number of operations to be performed on the double-linked list
- The next lines contain the integers separated by spaces in which the first integer indicates the operation to be performed and the second integer contains the element to be inserted or deleted.
- **1 --->** indicates the insertion
- **2 --->** indicates the deletion

Output Format:

- Display the elements of the double-linked list after performing the traversal operation.

Sample Test Case:

Input:

4
1 4
1 5
1 6
2 5

Output:

4 6

Explanation:

1 4 ---> 4 will be inserted
1 5 ---> 5 will be inserted
1 6 ---> 6 will be inserted
2 5 ---> 5 will be deleted
4 Operations are completed, The final output is the linked list ---> **4 6**

Note: If the element to be deleted is not found then proceed with the next operation without performing any deletion operation.

Program:

CTC17089.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

void insert(Node** head_ref, int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = NULL;

    if (*head_ref == NULL) {
        new_node->prev = NULL;
        *head_ref = new_node;
        return;
    }

    Node* temp = *head_ref;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = new_node;
    new_node->prev = temp;
}

void delete(Node** head_ref, int key) {
    Node* temp = *head_ref;

    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }

    if (temp == NULL) return;

    if (temp == *head_ref) {
        *head_ref = temp->next;
        if (*head_ref != NULL) {
            (*head_ref)->prev = NULL;
        }
        free(temp);
        return;
    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }

    free(temp);
}
```

CodeTantra

```
void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int operations;
    if (scanf("%d", &operations) != 1) return 0;

    Node* head = NULL;

    for (int i = 0; i < operations; i++) {
        int op, val;
        if (scanf("%d %d", &op, &val) != 2) return 0;

        if (op == 1) insert(&head, val);
        else if (op == 2) delete(&head, val);
    }

    printList(head);

    while (head != NULL) {
        Node* temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
4
1 4
1 5
1 6
2 5
4 6

Test case - 2
User Output
7
1 22
1 33

CodeTantra

2 44
1 55
1 66
2 55
1 77
22 33 66 77

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that uses functions to perform the following **operations** on a circularlinked list.

The operations performed on the circularlinked list are

1. Insertion
2. Deletion

Input format:

- The first line of input contains the number of operations to be performed on the circularlinked list
- The next lines contain the integers separated by spaces in which the first integer indicates the operation to be performed and the second integer contains the element to be inserted or deleted.
- **1 --->** indicates the insertion
- **2 --->** indicates the deletion

Output format:

- Display the elements of the circular linked list after performing all the operations, separated by spaces.

Sample Test Case:

Input:

4
1 4
1 5
1 6
2 5

Output:

4 6

Explanation:

1 4 ---> 4 will be inserted
1 5 ---> 5 will be inserted
1 6 ---> 6 will be inserted
2 5 ---> 5 will be deleted
4 Operations are completed, the final output is the linked list ---> **4 6**

Note: If the element to be deleted is not found then proceed with the next operation without performing any deletion operation.

Program:

CTC17088.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

void insert(Node** tail_ref, int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;

    if (*tail_ref == NULL) {
        new_node->next = new_node;
        *tail_ref = new_node;
    } else {
        new_node->next = (*tail_ref)->next;
        (*tail_ref)->next = new_node;
        *tail_ref = new_node;
    }
}

void delete(Node** tail_ref, int key) {
    if (*tail_ref == NULL) return;

    Node* curr = (*tail_ref)->next;
    Node* prev = *tail_ref;
    Node* start = curr;

    if (curr == *tail_ref && curr->data == key) {
        free(curr);
        *tail_ref = NULL;
        return;
    }

    do {
        if (curr->data == key) {
            if (curr == *tail_ref)
                *tail_ref = prev;
            prev->next = curr->next;
            free(curr);
            return;
        }
        prev = curr;
        curr = curr->next;
    } while (curr != start);
}

void printList(Node* tail) {
    if (tail == NULL) {
        printf("\n");
        return;
    }

    Node* curr = tail->next;
    do {
        printf("%d ", curr->data);
```

CodeTantra

```
        curr = curr->next;
    } while (curr != tail->next);
    printf("\n");
}

int main() {
    int n;
    scanf("%d", &n);

    Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        int op, val;
        scanf("%d %d", &op, &val);
        if (op == 1)
            insert(&tail, val);
        else if (op == 2)
            delete(&tail, val);
    }

    printList(tail);
    if (tail != NULL) {
        Node* curr = tail->next;
        Node* next;
        while (curr != tail) {
            next = curr->next;
            free(curr);
            curr = next;
        }
        free(tail);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
4
1 4
1 5
1 6
2 5
4 6

Test case - 2
User Output
7
1 22

CodeTantra

1 33
2 44
1 55
1 66
2 55
1 77
22 33 66 77

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that uses functions to perform the following operations on a **Doubly Circular Linked List**

1. Insertion
2. Deletion
3. Traversal

Input format:

- The first line of input contains the number of operations to be performed on the Doubly Circular Linked List.
- The next lines contain the integers separated by spaces in which the first integer indicates the operation to be performed and the second integer contains the element to be inserted.
- **1 --->** indicates the insertion
- **2 --->** indicates the deletion

Output format:

- Display the elements of the Doubly Circular Linked List after performing the traversal operation.

Note: The **deletion** operation is always performed on the last element.

Sample Test Case:

Input:

5
1 4
1 5
1 6
2
1 7

Output:

4 5 7

Explanation:

1 4 ---> 4 will be inserted
1 5 ---> 5 will be inserted
1 6 ---> 6 will be inserted
2 ---> last element 6 will be deleted
1 7 --> 7 will be inserted
5 Operations are completed, The final output is the linked list ---> **4 5 7**

Program:

CTC17090.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

void insert(Node** head_ref, int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;

    if (*head_ref == NULL) {
        new_node->next = new_node;
        new_node->prev = new_node;
        *head_ref = new_node;
        return;
    }

    Node* tail = (*head_ref)->prev;

    new_node->next = *head_ref;
    new_node->prev = tail;
    tail->next = new_node;
    (*head_ref)->prev = new_node;
}

void deleteLast(Node** head_ref) {
    if (*head_ref == NULL)
        return;

    Node* tail = (*head_ref)->prev;

     if (tail == *head_ref) {
         free(*head_ref);
        *head_ref = NULL;
        return;
    }

    Node* new_tail = tail->prev;
    new_tail->next = *head_ref;
    (*head_ref)->prev = new_tail;
    free(tail);
}

void printList(Node* head) {
    if (head == NULL) {
        printf("\n");
        return;
    }

    Node* temp = head;
    do {
        printf("%d ", temp->data);
        temp = temp->next;
```

CodeTantra

```
    } while (temp != head);
    printf("\n");
}

int main() {
    int n;
    scanf("%d",  &n);

    Node*  head = NULL;

    for (int i = 0; i < n; i++) {
        int op, val;
        scanf("%d",  &op);
        if (op == 1) {
            scanf("%d", &val);
            insert(&head, val);
        } else if (op == 2) {
            deleteLast(&head);
        }
    }

    printList(head);}
```

Output:

Test case - 1
User Output
5
1 4
1 5
1 6
2
1 7
4 5 7

Test case - 2
User Output
7
1 22
1 33
2
1 55
1 66
2
1 77
22 55 77

Result:

CodeTantra

Aim:

Franky is tasked with creating a one-way (singly) linked list. Once the linked list is created, he needs to **reverse** the linked list in-place. This means no extra memory (other than what is required for the linked list structure itself) should be used. Specifically, the space complexity must be **O(1)**, i.e., you cannot use extra data structures like arrays, lists, or stacks to store values during the reversal process. The reversal must modify the original linked list pointed to by the head pointer/reference.

Problem Details:

A linked list is a linear data structure where each node contains:

1. A value.
2. A pointer/reference to the next node in the list.

In this problem, you are required to reverse the linked list by modifying the next pointers of the nodes.

Input Format:

- First line: An integer N representing the number of nodes in the linked list.
- Second line: N space-separated integers, where each integer represents the value of a node in the linked list.

Output Format:

- Output the values of the linked list as space-separated integers in reverse order (after reversing the linked list).

Constraints:

- $1 \leq N \leq 10^3$
- $-10^6 \leq Data\ in\ Node \leq 10^6$
- Use the concept of a linked list to solve this problem.

Sample Test Case:

Input:

```
5
1 2 3 4 5
```

Output:

```
5 4 3 2 1
```

Instruction: To run your custom test cases, strictly map your input and output layout with the visible test cases.

Program:

```
CTC13633.c
```

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

 struct Node {
     int data;
    struct Node* next;
};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct Node* reverseList(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* curr = head;
    struct Node* next = NULL;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    return prev;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int n, value;
    scanf("%d", &n);

    if (n <= 0) return 0;

    scanf("%d", &value);
    struct Node* head = createNode(value);
    struct Node* current = head;

    for (int i = 1; i < n; i++) {
        scanf("%d", &value);
        current->next = createNode(value);
        current = current->next;
    }

    head = reverseList(head);

    printList(head);
```

CodeTantra

```
    while (head != NULL) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
5
1 2 3 4 5
5 4 3 2 1

Test case - 2
User Output
1
99
99

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a singly linked list of size M, and your task is to check whether the list is a **palindrome** or not.

Input format :

- The first line contains an Integer t which denotes the number of test cases or queries to be run.
- For each test case, you are given a singly linked list of integers where the list ends with -1. This -1 denotes the end of the list and is not part of the list.

Output format :

- For each test case, output **"true"** if the linked list is a palindrome, otherwise output **"false"**.

Constraints :

$1 \leq t \leq 10^2$

$1 \leq M \leq 10^3$ Where M is the size of the singly linked list.

Sample Test Case:

Input:

```
3
0 2 3 2 5 -1
1 0 1 -1
-1
```

Output:

```
false
true
true
```

Explanation:

- The list is [0, 2, 3, 2, 5], which is not a palindrome because it doesn't read the same forward and backward.
- The list is [1, 0, 1], which is a palindrome.
- The list is empty, so it is trivially considered as a palindrome.

Note: The outputs for all test cases should be printed at once, after processing all the queries, rather than immediately after each query. Please refer to the sample test cases for a better understanding of the output format.

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

Program:

CTC13059.c

Codetantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

 struct Node {
     int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct Node* reverseList(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* curr = head;
    struct Node* next = NULL;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

bool isPalindrome(struct Node* head) {
    if (head == NULL || head->next == NULL) {
        return true;
    }

    struct Node* slow = head;
    struct Node* fast = head;

    while (fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    struct Node* secondHalf = reverseList(slow->next);
    struct Node* firstHalf = head;

    struct Node* temp = secondHalf;
    bool isPalin = true;

    while (temp != NULL) {
        if (firstHalf->data != temp->data) {
            isPalin = false;
            break;
        }
        firstHalf = firstHalf->next;
        temp = temp->next;
    }
```

CodeTantra

```
    slow->next = reverseList(secondHalf);

    return isPalin;
}

void freeList(struct Node* head) {
    while (head != NULL) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    int t;
    scanf("%d",      &t);

    char results[t][6];

    for (int i = 0; i < t; i++) {
        struct Node* head = NULL;
        struct Node* tail = NULL;
        int val;

        while (1) {
            scanf("%d", &val);
            if (val == -1)
                break;
            struct Node* newNode = createNode(val);
            if (head == NULL) {
                head = newNode;
                tail = newNode;
            } else {
                tail->next = newNode;
                tail = tail->next;
            }
        }

        if (isPalindrome(head)) {
            sprintf(results[i], "true");
        } else {
            sprintf(results[i],  "false");
        }

        freeList(head);
    }
    for (int i = 0; i < t; i++) {
        printf("%s\n", results[i]);
    }

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
2
0 2 3 2 5 -1
-1
false
true

Test case - 2
User Output
1
9 2 3 3 2 9 -1
true

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Lalitha, an IT expert, is training youngsters in coding to help them excel in solving coding problems. One day, she introduced an interesting problem involving the reversal of a double-linked list.

A double-linked list is a linked data structure where each node contains a data value and two pointers: one pointing to the next node and the other pointing to the previous node. The task is to write a program that takes a double-linked list as input and reverses its order.

Input Format:

- The first line contains an integer N, representing the number of nodes in the double-linked list.
- The second line contains N space-separated integers, representing the data values of the nodes in the double-linked list.

Output Format:

- The program should return the reversed double-linked list by printing the data values of the nodes in the new order, separated by spaces.

Constraints:

- The number of nodes in the double-linked list (N) will be a positive integer.
- The data values of the nodes will be integers.

Example:

Input:

```
4
11 151 201 251
```

Output:

```
251 201 151 11
```

Explanation:

The initial double-linked list is 11 -> 151 -> 201 -> 251.

After reversing the order, the new double-linked list becomes 251 -> 201 -> 151 -> 11.

Program:

CTC31875.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

 struct Node {
     int data;
    struct Node* prev;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
struct Node* reverseDLL(struct Node* head) {
    struct Node* temp = NULL;
    struct Node* current = head;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        current = current->prev;
    }

    if (temp != NULL) {
        head = temp->prev;
    }

    return head;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int n;
    scanf("%d", &n);

     if (n <= 0) {
         return 0;
    }

    int val;
    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
```

CodeTantra

```
        struct Node* newNode = createNode(val);
        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }

    head = reverseDLL(head);

    printList(head);

    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
4
11 151 201 251
251 201 151 11

Test case - 2
User Output
5
1 2 3 4 5
5 4 3 2 1

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program to find the midpoint of a single linked list. If there are even number of nodes in the list, then print the second middle element.

Input Format:

- First line contains an integer, n, representing the number of elements of the list
- Second line contains n space separated integers representing the elements.

Output Format:

- Print the middle element of the single linked list.

Sample Test cases:

Input:

5
5 6 9 8 7

Output:

9

Input:

6
1 2 3 4 5 6

Output:

4

Program:

CTC26924.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

 struct Node {
     int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct Node* findMidpoint(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

int main() {
    int n, val;
    scanf("%d", &n);

    if (n <= 0) return 0;

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        struct Node* newNode = createNode(val);
        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = tail->next;
        }
    }

    struct Node* mid = findMidpoint(head);
    if (mid != NULL) {
        printf("%d\n", mid->data);
    }

    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
```

CodeTantra

```
        free(temp);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
5
5 6 9 8 7
9

Test case - 2
User Output
6
1 2 3 4 5 6
4

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given an array **Arr[]** of **N** no.of strings. Your job is to create a double linked list from the elements of the array and find the midpoint of the DLL

Note:

- For even no.of elements, consider the first node of the two middle nodes as the midpoint
- Elements of the array should be separated by ,(comma)

Complete the function **midPoint** with the following parameters:

- **N**: size of the Linked List
- **string Arr[]:**an array of strings

The function will return:

- **string:** the value of the midpoint

Constraints:

- $0 < N <= 10^5$
- $0 < len(Arr) < 10^5$

Sample Test Case

Input:

4

1,5,3,2

Output:

5

Important Instruction: Sample test case is for explanatory purposes but to run your custom test case on the terminal follow the input layout as mentioned in the command line arguments

Program:

CTC15005.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
    char *data;
    struct Node *next;
    struct Node *prev;
};

char * midPoint(int N, char *Arr[], int ArrLen) {
        if (ArrLen <= 0) {
        return "List is empty";
    }

    struct Node *head = NULL;
    struct Node *tail = NULL;

    for (int i = 0; i < ArrLen; i++) {
        struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            return "Memory Error";
        }
        newNode->data = Arr[i];
        newNode->next = NULL;
        newNode->prev = tail;

        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    struct Node *slow = head;
    struct Node *fast = head;

    while (fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    char* midpoint_data = slow->data;

    struct Node *current = head;
    while (current != NULL) {
        struct Node *temp = current;
        current = current->next;
        free(temp);
    }

    return midpoint_data;

}

int readStringArray(char *argsArray, char *arr[]) {
```

CodeTantra

```
        int col = 0;
        char *token = strtok(argsArray, ",");
        while (token != NULL) {
                arr[col] = token;
                token = strtok(NULL, ",");
                col++;
        }
        return col;
}
int main(int argc, char *argv[]) {
        int N = atoi(argv[1]);
        char *Arr[strlen(argv[2])];
        int ArrLen = readStringArray(argv[2], Arr);
        printf("%s\n", midPoint(N, Arr, ArrLen));
        return 0;
}
```

Output:

Test case - 1
User Output
5

Test case - 2
User Output
f

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given the heads of two sorted arrays/lists **list1[]** and **list2[]**.

Merge the two lists into one sorted list. This list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list

Note:

- Elements of the arrays should be ,(comma) separated.
- The final merged list should not be sorted.

Complete the function **mergeTwoSLL** with the following parameters:

integer list1[n1]: first array of n1 integers

integer list2[n2]: second array of n2 integers

Function will returns:

merged Single Linked List

Constraints:

- $0 < len(Arr1), len(Arr2) < 10^5$
- $0 < n1, n2 < 10^4$

Sample Test Case

Input 1:

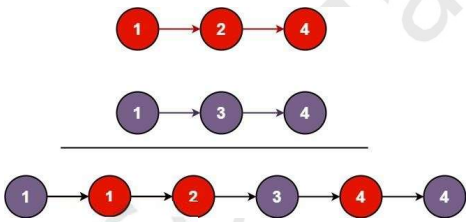
[1,2,4]

[1,3,4]

Output 1:

[1,1,2,3,4,4]

Explanation:



Input 2:

[ ]

[ ]

Output 2:

[ ]

Input 3:

[ ]

[0]

Output: [0]

Important Instruction : Sample test case is for explanatory purpose but to run your custom test case on the terminal follow the input layout as mentioned in the command line arguments

Program:

CTC15031.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MaxArrSize 100000


int mergeTwoSLL(short list1[], int list1Len, short list2[], int list2Len, short
*resultsArr) {
        int i = 0;
    int j = 0;
    int k = 0;

    while (i < list1Len && j < list2Len) {
        if (list1[i] <= list2[j]) {
            resultsArr[k++] = list1[i++];
        } else {
            resultsArr[k++] = list2[j++];
        }
    }

    while (i < list1Len) {
        resultsArr[k++] = list1[i++];
    }

    while (j < list2Len) {
        resultsArr[k++] = list2[j++];
    }

    return k;

}

int readShortArray(char *argsArray, short arr[]) {
        int col = 0;
        char *token = strtok(argsArray, ",");
        while (token != NULL) {
                arr[col] = (short) atoi(token);
                token = strtok(NULL, ",");
                col++;
        }
        return col;
}
void printArrayElements(short *resultsArr, int resultsArrLength) {
        int index;
        for(index = 0; index < resultsArrLength - 1; index++) {
                printf("%d,", resultsArr[index]);
        }
        printf("%d\n", resultsArr[index]);
}

int main(int argc, char *argv[]) {
        short list1[strlen(argv[1])];
        short list2[strlen(argv[2])];
        int list1Len = readShortArray(argv[1], list1);
        int list2Len = readShortArray(argv[2], list2);
        short resultsArr[MaxArrSize];
```

CodeTantra

```
        printArrayElements(resultsArr,  resultsArrLength);
        return 0;
}
```

Output:

Test case - 1
User Output
1,1,2,3,4,4

Test case - 2
User Output
-78,-40,-33,-21,-5,0,1,1,3,5,6,7,8,12,20,21,54,78

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given a linked list of**N**nodes. The task is to check if thelinked list has a loop. The linked list can containa self-loop.

Input format:

The first line contains an integer N, the number of nodes in the linked list
The Next line contains an array/list of integers separated by spaces.
The last line contains an integer X, the position at which the tail is connected.

Output format:

Print **True** if the linked list has a loop. Otherwise, print **False.**

Note: Position starts from 1

Constraints:

$1 \le N \le 10^3$
$1 \le$ Data on Node $\le 10^3$

Sample test case 1:

Input:

3
1 3 4
2

Output:

True

Explanation:

In the above test case N = 3. The linked list with nodes N = 3 is given. Then the value ofx = 2 is given which means the last node is connected with 2nd node of the linked list. Therefore, there exists a loop.

Sample test case 2:

Input:

7
1 5 3 6 4 8 9
0

Output:

False

Explanation:

For N = 7, x = 0 means then lastNode -> next = NULL, then theLinked list does not contain any loop.

Instructions:

- To run your custom test cases strictly map your input and output layout with the visible test cases.
- For **Java users** follow the instructions given in the below snapshot.

CodeTantra

q13373/CTJ13373.java

Java Submit

```java
// Java program written to solve the given problem.
package q13373; // CTJ13373.java must be saved inside the Package q13373.
import java.io.*;
import java.util.*;

class CTJ13373 { // class name must be same as file name CTJ13373.java.

    public static void main(String[] args) {
        // Write your driver code here.

    }

    // Write your other utility functions here.
}
```

Terminal Execution Results

Program:

CTC14238.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

 struct Node {
     int data;
    struct Node *next;
};

bool detectLoop(struct Node *head) {
    struct Node *slow_p = head, *fast_p = head;

    while (slow_p && fast_p && fast_p->next) {
        slow_p = slow_p->next;
        fast_p = fast_p->next->next;
        if (slow_p == fast_p) {
            return true;
        }
    }
    return false;
}

int main() {
    int N, data, X, i;
    struct Node *head = NULL, *tail = NULL;

    scanf("%d", &N);

    for (i = 0; i < N; i++) {
        scanf("%d", &data);
        struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) return 1;
        newNode->data = data;
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    scanf("%d", &X);

    if (X > 0 && X <= N) {
        struct Node *loopNode = head;
        for (i = 1; i < X; i++) {
            loopNode = loopNode->next;
        }
        if (tail != NULL) {
            tail->next = loopNode;
        }
    }

    bool hasLoop = detectLoop(head);
```

CodeTantra

```
    if(hasLoop)  {
        printf("True\n");
    } else {
        printf("False\n");
        struct Node *current = head;
        struct Node *next_node;
        while(current != NULL) {
            next_node = current->next;
            free(current);
            current = next_node;
        }
    }

    return 0;
}
```

Output:

Test case - 1
User Output
3
1 3 4
2
True

Test case - 2
User Output
7
1 5 3 6 4 8 9
0
False

CodeTantra

Result:

Thus the above program is executed successfully and the output has been verified

Aim:

Write a program to compute the Polynomial Addition d=a + b using singly linked list where a and b be the pointers to two polynomials.

Input Format

Number of Terms for the First Polynomial:

- An integer numTerms1 representing the number of terms in the first polynomial.

Terms of the First Polynomial:

- Followed by numTerms1 lines, each containing:
- An integer coeff (the coefficient of the term).
- An integer exp (the exponent of the term).

Number of Terms for the Second Polynomial:

- An integer numTerms2 representing the number of terms in the second polynomial.

Terms of the Second Polynomial:

- Followed by numTerms2 lines, each containing:
- An integer coeff (the coefficient of the term).
- An integer exp (the exponent of the term).

Output Format

Display of the First Polynomial:

- The polynomial is displayed in the format: coeff1x^exp1 + coeff2x^exp2 + ...
- Terms should be ordered by exponents in descending order.
- The format should include " + " between terms, but no trailing " + " at the end.

Display of the Second Polynomial:

- Similar format as the first polynomial: coeff1x^exp1 + coeff2x^exp2 + ...

Display of the Sum of the Two Polynomials:

- The resulting polynomial from adding the two polynomials, displayed in the format: coeff1x^exp1 + coeff2x^exp2 + ...

Note : Refer to sample test cases for better understanding

Program:

```
CTC39066.c
```

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int coeff;
    int exp;
    struct Node* next;
} Node;

Node* createNode(int coeff, int exp) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}

void insertTerm(Node** head, int coeff, int exp) {
    Node* newNode = createNode(coeff, exp);

    if (*head == NULL || (*head)->exp < exp) {
        newNode->next = *head;
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL && temp->next->exp > exp) {
            temp = temp->next;
        }
        if (temp->next != NULL && temp->next->exp == exp) {
            temp->next->coeff += coeff;
            free(newNode);
        } else {
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
}

void displayPolynomial(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%dx^%d", temp->coeff, temp->exp);
        temp = temp->next;
        if (temp != NULL)
            printf(" + ");
    }
    printf("\n");
}

Node* addPolynomials(Node* a, Node* b) {
    Node* result = NULL;
    Node *pa = a, *pb = b;

    while (pa != NULL && pb != NULL) {
        if (pa->exp > pb->exp) {
            insertTerm(&result, pa->coeff, pa->exp);
            pa = pa->next;
```

CodeTantra

```
        } else if (pa->exp < pb->exp) {
            insertTerm(&result, pb->coeff, pb->exp);
            pb = pb->next;
        } else {
            int sumCoeff = pa->coeff + pb->coeff;
            if (sumCoeff != 0)
                insertTerm(&result, sumCoeff, pa->exp);
            pa = pa->next;
            pb = pb->next;
        }
    }
    while (pa != NULL) {
        insertTerm(&result, pa->coeff, pa->exp);
        pa = pa->next;
    }
    while (pb != NULL) {
        insertTerm(&result, pb->coeff, pb->exp);
        pb = pb->next;
    }

    return result;
}

int main() {
    int numTerms1, numTerms2;
    int coeff, exp;
    Node* poly1 = NULL;
    Node* poly2 = NULL;
    scanf("%d", &numTerms1);
    for (int i = 0; i < numTerms1; i++) {
        scanf("%d %d", &coeff, &exp);
        insertTerm(&poly1, coeff, exp);
    }
    scanf("%d", &numTerms2);
    for (int i = 0; i < numTerms2; i++) {
        scanf("%d %d", &coeff, &exp);
        insertTerm(&poly2, coeff, exp);
    }

    displayPolynomial(poly1);
    displayPolynomial(poly2);

    Node* sum = addPolynomials(poly1, poly2);

    displayPolynomial(sum);

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
3

5 2
-3 1
1 0
3
2 3
4 1
-7 0
5x^2 + -3x^1 + 1x^0
2x^3 + 4x^1 + -7x^0
2x^3 + 5x^2 + 1x^1 + -6x^0

Test case - 2
User Output
4
2 4
-3 3
5 2
1 0
3
1 3
2 2
-7 1
2x^4 + -3x^3 + 5x^2 + 1x^0
1x^3 + 2x^2 + -7x^1
2x^4 + -2x^3 + 7x^2 + -7x^1 + 1x^0

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that implements the Stack Data structure using Arrays.

The operations performed on the Stack are

1. Push
2. Pop
3. Peek

Input Format:

1. The first line contains an integer n ($1 \le n \le 1000$), representing the number of operations to be performed on the stack.
2. The next n lines contain two integers separated by a space:
   - The first integer indicates the operation to be performed:
     - 1: Push an element onto the stack.
     - 2: Pop the top element from the stack.
     - 3: Peek the top element of the stack.
   - The second integer is only present if the operation is 1 (Push). It indicates the element to be pushed onto the stack.
   - If the operation is 2 (Pop) or 3 (Peek), there will be only one integer (the operation type) in the line.

Output Format:

- For every 3 (Peek) operation, print the top element of the stack on a new line.
- After all operations are performed, print the elements in the stack, space-separated.

Sample Test Case:

Input:

6
1 2
1 3
1 4
2
3
1 5

Output:

3
5 3 2

Explanation:

1 2 ---> 2 will be pushed
1 3 ---> 3 will be pushed
1 4 ---> 4 will be pushed
2 ---> Last element(4) is popped
3 ---> peek operation is performed which results in printing the top element of the stack i.e 3
1 5 ---> 5 will be pushed
6 Operations are completed
Therefore one peek operation is performed so the output is 3 and the final stack **5 3 2**

Note:

- If the stack is empty and when pop and peek are performed first, proceed to the next operation without displaying and modifying anything in the stack.
- If the stack is empty and nothing is pushed to the stack, Print Empty at the end.
- Display the output after all inputs have been taken.
- Refer to visible test cases to strictly match with input/output layout.

Program:

CTC17094.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 1000

int stack[MAX_SIZE];
int top = -1;

void push(int element) {
    if (top >= MAX_SIZE - 1) {
        return;
    }
    stack[++top] = element;
}

void pop() {
    if (top == -1) {
        return;
    }
    top--;
}

int peek() {
    if (top == -1) {
        return -1;
    }
    return stack[top];
}

int main() {
    int numOperations, operationType, element, peekResult;
    scanf("%d", &numOperations);

    int peekOutputs[MAX_SIZE];
    int numPeekOutputs = 0;

    for (int i = 0; i < numOperations; i++) {
        scanf("%d", &operationType);

        if (operationType == 1) {
            scanf("%d", &element);
            push(element);
        } else if (operationType == 2) {
            pop();
        } else if (operationType == 3) {
            peekResult = peek();
            if (peekResult != -1) {
                peekOutputs[numPeekOutputs++] = peekResult;
            }
        }
    }

    for (int i = 0; i < numPeekOutputs; i++) {
        printf("%d\n", peekOutputs[i]);
    }

    if (top == -1) {
```

CodeTantra

```
        printf("Empty\n");
    } else {
        for (int i = top; i >= 0; i--) {
            printf("%d", stack[i]);
            if (i > 0) {
                printf(" ");
            }
        }
        printf("\n");
    }

    return 0;
}
```

Output:

Test case - 1
User Output
6
1 2
1 3
1 4
2
3
1 5
3
5 3 2

Test case - 2
User Output
5
1 44
1 55
2
2
3
Empty

CodeTantra

Result:
Thus the above program is executed successfully and the output has been verified

Aim:

Write a program that implements the **Stack** Data structure using Linked lists.

The operations performed on the Stack are

1. Push
2. Pop
3. Peek

Input Format:

- The first line contains an integer n, the number of operations to be performed on the stack.
- The next n lines contain two integers each:
  - The first integer indicates the operation type (1 for Push, 2 for Pop, 3 for Peek).
  - If the operation is Push (indicated by 1), the second integer will be the element to be pushed onto the stack.
  - If the operation is Pop (indicated by 2), no second integer is needed.
  - If the operation is Peek (indicated by 3), no second integer is needed.

Output Format:

- After performing any Peek operation (operation type 3), print the top element of the stack.
- After performing all operations, print the elements of the stack, as shown in the sample test cases.

Sample Test Case:

Input:

```
6
1 2
1 3
1 4
2
3
1 5
```

Output:

```
3
5->3->2
```

Explanation:

1 2 ---> 2 will be pushed

1 3 ---> 3 will be pushed

1 4 ---> 4 will be pushed

2 ---> Last element(4) is popped

3 ---> peek operation is performed which results in printing the top element of the stack i.e 3

1 5 ---> 5 will be pushed

6 Operations are completed

Therefore one peek operation is performed so the output is 3 and the final stack **5->3->2**

Note:

- If the stack is empty and when pop and peek are performed first, proceed to the next operation without displaying and modifying anything in the stack.
- If the stack is empty and nothing is pushed to the stack, print "**Empty**" at the end.

Program:

CTC17095.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

 struct Node {
     int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int element) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        return;
    }
    newNode->data = element;
    newNode->next = top;
    top = newNode;
}

void pop() {
    if (top == NULL) {
        return;
    }
    struct Node *temp = top;
    top = top->next;
    free(temp);
}

int peek() {
    if (top == NULL) {
        return -1;
    }
    return top->data;
}
void printStack() {
    if (top == NULL) {
        printf("Empty\n");
        return;
    }

    struct Node *current = top;
    while (current != NULL) {
        printf("%d", current->data);
        if (current->next != NULL) {
            printf("->");
        }
        current = current->next;
    }
    printf("\n");
}

int main() {
    int numOperations, operationType, element;
    scanf("%d", &numOperations);

    int peekOutputs[numOperations];
```

CodeTantra

```
    int numPeekOutputs = 0;

    for (int i = 0; i < numOperations; i++) {
        scanf("%d", &operationType);

        if (operationType == 1) {
            scanf("%d", &element);
            push(element);
        } else if (operationType == 2) {
            pop();
        } else if (operationType == 3) {
            int peekResult = peek();
            if (peekResult != -1) {
                peekOutputs[numPeekOutputs++] = peekResult;
            }
        }
    }

    for (int i = 0; i < numPeekOutputs; i++) {
        printf("%d\n", peekOutputs[i]);
    }

    printStack();

    while (top != NULL) {
        pop();
    }

    return 0;
}
```

Output:

Test case - 1
User Output
6
1 2
1 3
1 4
2
3
1 5
3
5->3->2

Test case - 2
User Output
5
1 44
1 55

CodeTantra

2
3
Empty

Test case - 3
User Output
7
1 7
1 8
2
1 9
3
1 10
2
9
9->7

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a string expr which contains an infix arithmetic expression. Your task is to convert the infix expression to an equivalent postfix expression using a stack-based approach.

Function Specification:

- Complete the function InfixToPostfix with the following parameter:
  - **expr** (string): The infix expression containing operands (single letters or digits), operators (+, -, *, /, ^), and parentheses (,).
- The function should return:
- string: The postfix expression equivalent to the given infix expression.

Input Format:

- The input is given as a command-line argument, which contains the infix expression.

Output Format:

- Return the postfix expression as a string.

Constraints:

- 0 < len(expr) <= 10^5
- The expression contains only valid single-character operands, operators, and parentheses.
- Exponentiation (^) is right-associative, and the usual operator precedence applies: (^ > * / > + -).

Sample Test Case:

Input: (a+b/c)

Output: abc/+

Note: The Sample test case is for explanatory purposes, but to run your custom test case on the terminal, follow the input layout as mentioned in the command line arguments.

Program:

CTC9758.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_EXPR_SIZE 100005
char stack[MAX_EXPR_SIZE];
int top = -1;

void push(char item) {
    if (top >= MAX_EXPR_SIZE - 1) {
        return;
    }
    stack[++top] = item;
}

char pop() {
    if (top < 0) {
        return '\0';
    }
    return  stack[top--];
}

char peek() {
    if (top < 0) {
        return '\0';
    }
    return stack[top];
}

int isEmpty() {
    return top == -1;
}

int isOperand(char ch) {
    return isalnum(ch);
}

int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
    }
    return -1;
}
char * InfixToPostfix(char *expr) {
        top = -1;
    int i;
    int k = 0;
    int len = strlen(expr);
```

CodeTantra

```
    char *postfix = (char*)malloc((len + 1) * sizeof(char));
    if (!postfix) {
        return NULL;
    }
    for (i = 0; i < len; i++) {
        char currentChar = expr[i];

        if (isOperand(currentChar)) {
            postfix[k++] = currentChar;
        }
        else if (currentChar == '(') {
            push(currentChar);
        }
        else if (currentChar == ')') {
            while (!isEmpty() && peek() != '(') {
                postfix[k++] = pop();
            }
            pop();
        }
        else {
            if (currentChar == '^') {
                 while (!isEmpty() && peek() != '(' && precedence(currentChar) <
precedence(peek())) {
                    postfix[k++] = pop();
                }
            } else {
                 while (!isEmpty() && peek() != '(' && precedence(currentChar) <=
precedence(peek())) {
                    postfix[k++] = pop();
                }
            }
            push(currentChar);
        }
    }

    while (!isEmpty()) {
        postfix[k++] = pop();
    }

    postfix[k] = '\0';

    return postfix;

}

int main(int argc, char *argv[]) {
        char *expr = argv[1];
        printf("%s\n", InfixToPostfix(expr));
        return 0;
}
```

Output:

Test case - 1
User Output

CodeTantra

abc/+

Test case - 2
User Output
abc/d*+ef^-

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a string **str** of some set of braces**.** Your task is to check whether the given parentheses in the string are balanced or not.

Complete the function **BalancedBrackets** with the following parameter(s):

string *str*:String that contains the parenthesis.

The functionreturns:

A **string** that returns'**Balanced**' if the parenthesis of the input string are balanced or returns '**Not balanced**' if the parenthesis are not balanced.

Constraints:

- **1 ≤length of each string str ≤ 10^2**

Note: Every string consists of (,), {, }, [, and] braces only.

Sample test case 1:

Input:

```
[{}]{}[]()
```

Output:

```
Balanced
```

Explanation:

The brackets in the first '**[{}]{}[]()**' are balanced, because all brackets are closed and all nested brackets are closed in order. So the output is **Balanced.**

Sample test case 2:

Input:

```
[{]}
```

Output:

```
Not balanced
```

Explanation:

The brackets in the second string '**[{]}**' are not balanced, because the nested bracket '{' was not closed before it's surrounding '[ ]', so the order was not respected. Then the output is **Not balanced.**

Instructions: To run your custom test cases strictly map your input and output layout with the visible test cases.

Program:

CTC15014.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include  <string.h>

char * BalancedBrackets(char *str) {
        int n = strlen(str);
    char stack[110];
    int top = -1;

    for (int i = 0; i < n; i++) {
        if (str[i] == '(' || str[i] == '{' || str[i] == '[') {
            stack[++top] = str[i];
        }
        else if (str[i] == ')' || str[i] == '}' || str[i] == ']') {
            if (top == -1) {
                return "Not balanced";
            }

            char last = stack[top--];
            if ((str[i] == ')' && last != '(') ||
                (str[i] == '}' && last != '{') ||
                (str[i] == ']' && last != '[')) {
                return "Not balanced";
            }
        }
    }
    return (top == -1) ? "Balanced" : "Not balanced";

}

int main(int argc, char *argv[]) {
        char *str = argv[1];
        printf("%s\n", BalancedBrackets(str));
        return 0;
}
```

Output:

Test case - 1
User Output
Balanced

Test case - 2
User Output
Not balanced

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Your task is to design a program that reverse the order of elements in a stack using recursion, without utilizing any loops.

Input format:

- The first line should contain an integer n representing the number of elements in the stack.
- The second line should contain n space-separated integers representing the elements of the stack.

Output format:

- The output will display the elements of the reversed stack, also separated by space

Constraints:

- $1 \leq$ *size of the stack* $\leq 10^4$
- $-10^9 \leq$ *Each element of the stack* $\leq 10^9$

Example input :

```
8
2 9 -54 92 6 9 2 0
```

Example output:

```
0 2 9 6 92 -54 9 2
```

Program:

CTC30989.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100
int stack[MAX_SIZE];
int top = -1;

int isEmpty() {
    return top == -1;
}
void push(int item) {
    if (top < MAX_SIZE - 1) {
        stack[++top] = item;
    }
}
int pop() {
    if (!isEmpty()) {
        return stack[top--];
    }
    return -1;
}
void print_stack_recursively() {
    if (isEmpty()) {
        return;
    }
    int item = pop();
    printf("%d", item);
    if (!isEmpty()) {
        printf(" ");
    }
    print_stack_recursively();
}

int main() {
    int n, i, element;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &element);
        push(element);
    }
    print_stack_recursively();
    printf("\n");

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
8
2 9 -54 92 6 9 2 0
0 2 9 6 92 -54 9 2

Test case - 2
User Output
10
-5 -6 79 373 9872 82 9 102 523 0
0 523 102 9 82 9872 373 79 -6 -5

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program to check whether the given element is present or not in the array of elements using the binary search Technique

Input Format:

- The first line of input contains an integer N representing the no. of elements of the array
- The second line input contains the array of N integers separated by a space
- The last line of input contains the key element to be searched

Output Format:

- If the element is found, print the index.
- If the element is not found, print "Not found".

Sample Test Case:

Input:

```
7
1 2 3 4 3 5 6
3
```

Output:

```
2
```

Note:

- If two duplicate elements are present, then it should print the first element's index value.
- Refer to the visible test cases for better understanding.

Program:

CTC17128.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

int linear_search(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int n;
    scanf("%d", &n);
    int *arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int key;
    scanf("%d", &key);
    int result_index = linear_search(arr, n, key);
    if (result_index != -1) {
        printf("%d\n", result_index);
    } else {
        printf("Not found\n");
    }
    free(arr);

    return 0;
}
```

Output:

Test case - 1
User Output
7
1 2 3 4 3 5 6
3
2

Test case - 2
User Output
10
1 2 3 4 5 6 7 8 9 19
20
Not found

CodeTantra

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Problem:

You are given a positive integer **N**. Your job is to print the Fibonacci series of numbers as an array/list up to the given integer **N**.

Complete the function **fibonacciNumbers** with the following parameter:
integer N

Function will returns:
Array: an array of integers that is having all the fibonacci numbers upto N

Constraints:
$0 < N <= 10^5$

Sample Test Case
Input:
5

Output:
[0, 1, 1, 2, 3, 5]

Program:

```
CTC9250.c
```

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

int fibonacciNumbers(int N, int *resultsArr) {
    int count = 0;
    long long a = 0, b = 1;
    if (N >= 0) {
        resultsArr[count] = 0;
        count++;
    } else {
        return 0;
    }
    if (N >= 1) {
        resultsArr[count] = 1;
        count++;
    }
    while (1) {
        long long next = a + b;

        if (next > N) {
            break;
        }
        resultsArr[count] = (int)next;
        count++;
        a = b;
        b = next;
    }
    return count;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <N>\n", argv[0]);
        return 1;
    }
    int N = atoi(argv[1]);

    int resultsArr[100];
    int count;
    count = fibonacciNumbers(N, resultsArr);
    for (int i = 0; i < count; i++) {
        printf("%d", resultsArr[i]);
        if (i < count - 1) {
            printf(",");
        }
    }
    printf("\n");

    return 0;
}
```

Output:

Test case - 1
User Output

CodeTantra

0,1,1,2,3,5

Test case - 2
User Output
0,1,1,2,3,5,8,13,21,34,55,89

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given a Towers of Hanoi puzzle with N discs initially placed on $Peg\ A$, you need to write a function to find the minimum number of movements required to solve the puzzle, moving all the discs from $Peg\ A$ to $Peg\ C$, using $Peg\ B$ as an intermediate Peg.

Input Format:

- The input consists of a single integer N, representing the number of discs.

Output Format:

- The function should return an integer representing the minimum number of movements required to solve the puzzle.

Constraints:

- $1 \leq N \leq 20$

Example:

Input:

```
N = 3
```

Output:

```
7
```

Explanation:

- For a Towers of Hanoi puzzle with 3 discs, it takes a minimum of 7 movements to move all the discs from $Peg\ A$ to $Peg\ C$ using $Peg\ B$ as an intermediate peg.

Note:

- In the Towers of Hanoi puzzle, the number of movements required to solve the puzzle follows the pattern $2^N - 1$, where N is the number of discs.

Instruction:

- To run your custom test cases, strictly map your input and output layout with the visible test cases.

Program:

CTC17071.c

```c
#include <stdio.h>

long long calculate_hanoi_moves(int n) {
    return (1LL << n) - 1;
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%lld\n", calculate_hanoi_moves(n));

    return 0;
}
```

Output:

Test case - 1
User Output
3
7

CodeTantra

Test case - 2
User Output
5
31

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that implements the Queue Data structure using Arrays

The operations to be performed on the Queue are

1. Enqueue
2. Dequeue

Input format:

- The first line contains a single integer n, which denotes the number of operations to be performed on the queue.
- Each of the following n lines contains two space-separated integers:
- The first integer indicates the operation:
  - 1 → Enqueue operation (followed by an element to be added to the queue).
  - 2 → Dequeue operation (removes the front element of the queue).
  - If the operation is 1, the second integer is the element to be enqueued.
  - If the operation is 2, no second integer follows since it's a dequeue operation.

Output format:

- Display the queue with space separated integers after performing all operations at the end.

Sample Test Case:

Input:

6
1 2
1 3
1 4
2
2
1 5

Output:

4 5

Explanation:

1 2 ---> 2 will be inserted
1 3 ---> 3 will be inserted
1 4 ---> 4 will be inserted
2 ---> first element(2) is removed
2 ---> second element(3) is removed
1 5 ---> 5 will be inserted
6 Operations are completed
The final queue is **4 5**

Note:

- If the queue is empty till the end and nothing is inserted into the queue, Print **Empty** at the end.
- If the queue is empty and when dequeue is performed first, proceed to the next operation without displaying and modifying anything in the queue.
- Display the output after all inputs have been taken.
- Refer to the displayed test cases for better understanding.

Program:

CTC17096.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int queue[MAX_SIZE];
int front = 0;
int rear = -1;
void enqueue(int value) {
    if (rear >= MAX_SIZE - 1) {
        return;
    }
    rear++;
    queue[rear] = value;
}
void dequeue() {
    if (rear < front) {
        return;
    }
    front++;
}
void printQueue() {
    if (rear < front) {
        printf("Empty\n");
        return;
    }
    for (int i = front; i <= rear; i++) {
        printf("%d", queue[i]);
        if (i < rear) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    int n;
    scanf("%d", &n);

    int op_type, value;
    for (int i = 0; i < n; i++) {
        scanf("%d", &op_type);

        if (op_type == 1) {
            scanf("%d", &value);
            enqueue(value);
        } else if (op_type == 2) {
            dequeue();
        }
    }
    printQueue();

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
6
1 2
1 3
1 4
2
2
1 5
4 5

Test case - 2
User Output
3
1 11
2
2
Empty

Test case - 3
User Output
5
2
1 33
1 44
1 55
2
44 55

CodeTantra

Result:

Thus the above program is executed successfully and the output has been verified

Aim:

Write a program that implements the **Circular Queue** using Arrays

The operations to be performed on the **Circular Queue** are

1. Enqueue
2. Dequeue

Input Format:

- The first line of input contains an integer n, representing the number of operations to be performed on the Circular Queue.
- The next n lines contain the integers separated by a space, in which the first integer indicates the operation to be performed and the second integer contains the element to be enqueued.
  - **1 --->** indicates the Enqueue
  - **2 --->** indicates the Dequeue

Output Format:

- Display the Circular Queue after performing all operations at the end.

Sample Test Case:

Input:

6
1 2
1 3
1 4
2
2
1 5

Output:

4 5

Explanation:

1 2 ---> 2 will be inserted
1 3 ---> 3 will be inserted
1 4 ---> 4 will be inserted
2 ---> first element(2) is removed
2 ---> second element(3) is removed
1 5 ---> 5 will be inserted
6 Operations are completed
The final Circular Queue is **4 5**

Note:

- If the queue is empty till the end and nothing is inserted into the queue, print "Empty" at the end.
- If the queue is empty and the dequeue is performed first, proceed to the next operation without displaying or modifying anything in the Circular Queue.

Program:

CTC17098.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int queue[MAX_SIZE];
int front = -1;
int rear = -1;
void enqueue(int value) {
    if ((rear + 1) % MAX_SIZE == front) {
        return;
    }
     if (front == -1) {
         front = 0;
         rear = 0;
     } else {
         rear = (rear + 1) % MAX_SIZE;
    }
    queue[rear] = value;
}
void dequeue() {
    if (front == -1) {
        return;
    }
    if (front == rear) {
        front = -1;
        rear = -1;
    } else {
        front = (front + 1) % MAX_SIZE;
    }
}
void printQueue() {
    if (front == -1) {
        printf("Empty\n");
        return;
    }

    int i;
    for (i = front; i != rear; i = (i + 1) % MAX_SIZE) {
        printf("%d ", queue[i]);
    }
    printf("%d\n", queue[i]);
}

int main() {
    int n;
    scanf("%d", &n);

    int op_type, value;
    for (int i = 0; i < n; i++) {
        scanf("%d", &op_type);

        if (op_type == 1) {
            scanf("%d", &value);
            enqueue(value);
        } else if (op_type == 2) {
            dequeue();
```

CodeTantra

```
        }
    }
    printQueue();

    return 0;
}
```

Output:

Test case - 1
User Output
6
1 2
1 3
1 4
2
2
1 5
4 5

Test case - 2
User Output
3
1 11
2
2
Empty

Test case - 3
User Output
5
2
1 33
1 44
1 55
2
44 55

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program that implements the **Queue** Data structure using two stacks

The operations to be performed on the Queue are

1. Enqueue
2. Dequeue

Input format:

- The first line of input contains the number of operations to be performed on the queue
- The next lines contain the integers separated by space in which the first integer indicates the operation to be performed and the second integer contains the element to be enqueued.
- **1 --->** indicates the Enqueue
- **2 --->** indicates the Dequeue

Output format:

- Display the queue after performing all operations at the end.

Sample Test Case:

Input:

6
1 2
1 3
1 4
2
2
1 5

Output:

4 5

Explanation:

1 2 ---> 2 will be inserted
1 3 ---> 3 will be inserted
1 4 ---> 4 will be inserted
2 ---> first element(2) is removed
2 ---> second element(3) is removed
1 5 ---> 5 will be inserted
6 Operations are completed
The final queue is **4 5**

Note:

- If the queue is empty till the end and nothing is inserted into the queue, Print **Empty** at the end.
- If the queue is empty and when dequeue is performed first, proceed to the next operation without displaying and modifying anything in the queue.

Program:

CTC39024.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int stack1[MAX_SIZE];
int top1 = -1;

int stack2[MAX_SIZE];
int top2 = -1;

void push1(int data) {
    if (top1 < MAX_SIZE - 1) {
        stack1[++top1] = data;
    }
}

int pop1() {
    if (top1 >= 0) {
        return stack1[top1--];
    }
    return -1;
}

int isEmpty1() {
    return top1 == -1;
}
void push2(int data) {
    if (top2 < MAX_SIZE - 1) {
        stack2[++top2] = data;
    }
}

int pop2() {
    if (top2 >= 0) {
        return stack2[top2--];
    }
    return -1;
}

int isEmpty2() {
    return top2 == -1;
}
void enqueue(int data) {
    push1(data);
}
void dequeue() {
    if (isEmpty2()) {
        while (!isEmpty1()) {
            push2(pop1());
        }
    }
    if (!isEmpty2()) {
        pop2();
    }
}
void printQueue() {
```

CodeTantra

```
    if (isEmpty1() && isEmpty2()) {
        printf("Empty\n");
        return;
    }

    int i;
    int first_element_printed = 0;
    for (i = top2; i >= 0; i--) {
        if (first_element_printed) {
            printf(" ");
        }
        printf("%d", stack2[i]);
        first_element_printed = 1;
    }
    for (i = 0; i <= top1; i++) {
        if (first_element_printed) {
            printf(" ");
        }
        printf("%d", stack1[i]);
        first_element_printed = 1;
    }

    printf("\n");
}


int main() {
    int n, op, value;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &op);
        if (op == 1) {
            scanf("%d", &value);
            enqueue(value);
        } else if (op == 2) {
            dequeue();
        }
    }
    printQueue();

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
6
1 2
1 3
1 4
2
2

4 5

Test case - 2
User Output
3
1 11
2
2
Empty

Test case - 3
User Output
5
2
1 33
1 44
1 55
2
44 55

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Write a program to implement the Priority Queue using Heaps. The following operations are to be performed.

1. Enqueue
2. Dequeue

Input format:

- The first line of input contains the number of operations to be performed on the Priority Queue
- The next lines contain the integers separated by space in which the first integer indicates the operation to be performed and the second integer indicates the element to be inserted and the third integer is the respective priority (**Only for insertion**)
- **1 --->** indicates the Enqueue
- **2 --->** indicates the Dequeue

Output format:

- Display the Priority queue after performing all the operations.

Sample Test Case:

Input:

4
1 4 5
1 7 5
1 2 3
2

Output:

4 (5) 7 (5)

Explanation:

1 4 5 --> 4 will be inserted with the priority 5
1 7 5 --> 7 will be inserted with the priority 5
1 2 3 --> 2 will be inserted with the priority 3
2 --> Element (2) with the most priority will be removed.
Finally, the priority Queue is displayed.

Note:

- If the queue is empty and when dequeue is performed first, proceed to the next operation without displaying and modifying anything in the queue.
- If the queue is empty till the end and nothing is inserted into the queue, Print **Empty** at the end.
- Refer to the test cases for a better understanding

Program:

CTC17101.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct {
    int key;
    int value;
    int time;
} Element;

Element* pq = NULL;
int pq_size = 0;
int pq_capacity = 0;
int time_counter = 0;

void insert_or_update(int key, int value) {
    for (int i = 0; i < pq_size; i++) {
        if (pq[i].key == key) {
            pq[i].value = value;
            return;
        }
    }
    if (pq_size == pq_capacity) {
        pq_capacity = (pq_capacity == 0) ? 10 : pq_capacity * 2;
        pq = realloc(pq, pq_capacity * sizeof(Element));
        if (pq == NULL) { exit(1); }
    }
    pq[pq_size].key = key;
    pq[pq_size].value = value;
    pq[pq_size].time = time_counter++;
    pq_size++;
}
void delete_min() {
    if (pq_size == 0) {
        return;
    }

    int min_val = INT_MAX;
    int min_time = INT_MAX;
    int min_idx = -1;
    for (int i = 0; i < pq_size; i++) {
        if (pq[i].value < min_val) {
            min_val = pq[i].value;
            min_time = pq[i].time;
            min_idx = i;
        } else if (pq[i].value == min_val) {
            if (pq[i].time < min_time) {
                min_time = pq[i].time;
                min_idx = i;
            }
        }
    }
    if (min_idx != -1) {
        pq[min_idx] = pq[pq_size - 1];
        pq_size--;
    }
}
```

CodeTantra

```
int compare_elements(const void* a, const void* b) {
    Element* elemA = (Element*)a;
    Element* elemB = (Element*)b;
    if (elemA->value != elemB->value) {
        return elemA->value - elemB->value;
    }
    return elemA->key - elemB->key;
}
void print_pq() {
    if (pq_size == 0) {
        printf("Empty\n");
        return;
    }

    qsort(pq, pq_size, sizeof(Element), compare_elements);

    for (int i = 0; i < pq_size; i++) {
        printf("%d (%d)", pq[i].key, pq[i].value);
        if (i < pq_size - 1) {
            printf(" ");
        }
    }
    printf("\n");
}


int main() {
    int n, op, key, value;
    if (scanf("%d", &n) != 1) return 1;

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &op) != 1) return 1;
        if (op == 1) {
            if (scanf("%d %d", &key, &value) != 2) return 1;
            insert_or_update(key, value);
        } else if (op == 2) {
            delete_min();
        }
    }

    print_pq();
    if (pq != NULL) {
        free(pq);
    }

    return 0;
}
```

Output:

Test case - 1
User Output
4
1 4 5

CodeTantra

1 7 5
1 2 3
2
4 (5) 7 (5)

Test case - 2
User Output
7
2
1 15 5
1 20 7
2
1 3 2
1 4 2
2
4 (2) 20 (7)

Test case - 3
User Output
4
1 3 3
2
1 4 5
2
Empty

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are required to implement a double-ended queue (deque) using a linked list in C. Your task is to provide the implementation for the following operations:

1. **Insert at Front**: Add an element to the front of the deque.
2. **Insert at Rear**: Add an element to the rear of the deque.
3. **Delete from Front**: Remove an element from the front of the deque.
4. **Delete from Rear**: Remove an element from the rear of the deque.
5. **Display**: Print all elements of the deque from front to rear.
6. **Exit**: Terminate the program.

Note: Refer to the displayed test cases for better understanding.

Program:

```
dooublyLinkedList.c
```

CodeTantra

```c
#include<stdio.h>
#include<stdlib.h>

struct node{
        int data;
        struct node *next;
};
typedef struct node * NODE;
NODE rear=NULL,front=NULL;

NODE createNode() {
        NODE newNode = (NODE)malloc(sizeof(struct node));
        if(newNode == NULL){
                printf("Memory allocation failed\n");
                exit(0);
        }
        newNode->next = NULL;
        return newNode;
}

void insertRear(int x) {
        NODE newNode = createNode();
        newNode->data = x;
        if(front == NULL) {
                front = rear = newNode;
                return;
        }
        rear->next = newNode;
        rear = newNode;
}

void insertFront(int x) {
        NODE newNode = createNode();
        newNode->data = x;
        if(front == NULL) {
                front = rear = newNode;
                return;
        }
        newNode->next = front;
        front = newNode;
}

void deleteFront() {
        if(front == NULL) {
                printf("Deque is empty\n");
                return;
        }

        NODE temp = front;
        front = front->next;
        if(front == NULL) {
                rear = NULL;
        }
        printf("The deleted element from Front : %d\n",temp->data);
    free(temp);
}
```

CodeTantra

```
void deleteRear(){
        if(front == NULL) {
                printf("Deque is empty\n");
                return;
        }

    NODE temp;
        if(front == rear) {
                temp = front;
                front = rear = NULL;
        } else {
                NODE second_last = front;
                while(second_last->next != rear) {
                        second_last  =  second_last->next;
                }

                temp = rear;
                rear = second_last;
                rear->next = NULL;
        }
        printf("The deleted element from Rear : %d\n",temp->data);
    free(temp);
}

void print()
{
        NODE temp=front;
        while(temp)
        {
                printf("%d->",temp->data);
                temp=temp->next;
        }
        printf("NULL\n");
}

void main()
{
        int data,choice;
        while(1)
        {
                printf("1.Insert at Front\n2.Insert at Rear\n3.Delete from
Front\n4.Delete from Rear\n5.Display\n6.Exit\n");
                printf("Enter your choice:");
                printf("\n");
                scanf("%d",&choice);
                switch(choice)
                {
                case 1:
                        printf("Enter an element to Insert at Front:");
                        scanf("%d",&data);
                        insertFront(data);
                        break;
                case 2:
                    printf("Enter an element to Insert at Rear:");
                        scanf("%d",&data);
                        insertRear(data);
                        break;
```

CodeTantra

```
        case 3:
                        if(front==NULL)
                    printf("Deque is empty\n");
                else
                            deleteFront();
                        break;
                    case 4:
                        if(rear==NULL)
                    printf("Deque is empty\n");
                else
                            deleteRear();
                        break;
                    case 5:
                        if(front==NULL)
                     printf("Deque is empty\n");
                else
                                print();
                        break;
                    case 6:
                        exit(0);
                    default:
                        printf("\nEnter a valid choice");
                        break;
                    }
        }
}
```

Output:

Test case - 1
User Output
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice:
5
Deque is empty
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice:
3
Deque is empty
1. Insert at Front
2. Insert at Rear
3. Delete from Front

CodeTantra

```
6.Exit
Enter your choice:
4
Deque is empty
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
1
Enter an element to Insert at Front:
3
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
5
3->NULL
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
2
Enter an element to Insert at Rear:
5
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
5
3->5->NULL
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
3
The deleted element from Front : 3
```

CodeTantra

```
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
5
5->NULL
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
4
The deleted element from Rear : 5
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
5
Deque is empty
1.Insert at Front
2.Insert at Rear
3.Delete from Front
4.Delete from Rear
5.Display
6.Exit
Enter your choice:
6
```

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given an array/list **Arr[]** of strings. You have to create a doubly linked list using the elements of the array/list and sort the elements of the linked list using merge sort.

Note: Data of sorted doubly linked list must be returned as a string in required format mentioned in the visible test cases.

Complete the function **mergeSort** with the following parameters:
string Arr[n]: an array of n strings
Function will return:
string: string of the merged list as shown in the output

Constraints:

- $0 < len(Arr) < 10^5$
- string data in the linked list can have numeric characters or alphabets.
- Numeric characters or aphabets must be sorted in lexicographical order.

Sample Test Case
Input:
orange,red,blue,green,yellow

Output:
blue -> green -> orange -> red -> yellow (-> is used to represent doubly linked list link between two nodes, format is blue{space}->{space} .....)

Instruction: For input and output layout strictly follow the visible sample test cases and follow the function rules mentioned below.

Program:

CTC9272.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node {
    char *data;
    struct Node *next;
    struct Node *prev;
} Node;

Node *createNode(char *data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = strdup(data);
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

Node *merge(Node *left, Node *right) {
    if (!left) return right;
    if (!right) return left;

    Node *result = NULL;

    if (strcmp(left->data, right->data) <= 0) {
        result = left;
        result->next = merge(left->next, right);
        if (result->next) result->next->prev = result;
    } else {
        result = right;
        result->next = merge(left, right->next);
        if (result->next) result->next->prev = result;
    }

    return result;
}

Node *mergeSortList(Node *head) {
    if (!head || !head->next) return head;

    Node *slow = head;
    Node *fast = head->next;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    Node *midNext = slow->next;
    slow->next = NULL;
    if (midNext) midNext->prev = NULL;

    Node *left = mergeSortList(head);
    Node *right = mergeSortList(midNext);

    return merge(left, right);
}
```

CodeTantra

```
char *linkedListToString(Node *head) {
    char *result = (char *)malloc(1);
    result[0] = '\0';
    Node *current = head;

    while (current) {
        char *temp;
        if (strlen(result) == 0) {
            asprintf(&temp, "%s", current->data);
        } else {
            asprintf(&temp, "%s -> %s", result, current->data);
            free(result);
        }
        result = temp;
        current = current->next;
    }

    return result;
}
char * mergeSort(char *Arr[], int ArrLen) {
        Node *head = NULL;
    Node *tail = NULL;

    for (int i = 0; i < ArrLen; i++) {
        Node *newNode = createNode(Arr[i]);
        if (!head) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }

    head = mergeSortList(head);

    char *result = linkedListToString(head);

    Node *current = head;
    while (current) {
        Node *temp = current;
        current = current->next;
        free(temp->data);
        free(temp);
    }

    return result;

}

int readStringArray(char *argsArray, char *arr[]) {
        int col = 0;
        char *token = strtok(argsArray, ",");
        while (token != NULL) {
                arr[col] = token;
                token = strtok(NULL, ",");
```

CodeTantra

```
                col++;
        }
        return col;
}
int main(int argc, char *argv[]) {
        char *Arr[strlen(argv[1])];
        int ArrLen = readStringArray(argv[1], Arr);
        printf("%s\n", mergeSort(Arr, ArrLen));
        return 0;
}
```

Output:

Test case - 1
User Output
blue -> green -> orange -> red -> yellow

Test case - 2
User Output
apple

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given a list of N array elements, apply **quick sort** on an array using recursion to arrange the elements in **ascending order**.

Constraints:

- $1 <= N <= 1000$
- $-100000 <= a[i] <= 100000$

Input Format:

- The first line contains the integer N, the size of the array.
- The next line contains N space-separated integers.

Output Format:

- Print the sorted array as a row of space-separated integers.

Note: Use the divide and conquer using the recursion approach to solve the question.

Program:

```
CTC38656.c
```

CodeTantra

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
         if (i < n - 1) {
             printf(" ");
        }
    }
    printf(" \n");
    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output

5
4 8 2 -3 0
-3 0 2 4 8

Test case - 2
User Output
8
-6 -2 -8 -36 0 69 -5 -96
-96 -36 -8 -6 -5 -2 0 69

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are tasked with managing the scores of students from two different tests. The scores for each test are stored in two separate arrays ($nums1$ and $nums2$), which are both sorted in non-decreasing order. The challenge is to merge these two sorted arrays into a single sorted array.

In particular, you are given:

1. An array $nums1$ of size $m + n$, where the first m elements contain the scores from the first test and the remaining n elements are initialized to 0. These 0s are placeholders to make room for the elements of the second array $nums2$.
2. An array $nums2$ of size n, which contains the scores from the second test.
3. Your goal is to merge these two arrays into one sorted array, placing the result in $nums1$, so that the final array contains all the scores from both tests, sorted in non-decreasing order.
4. You need to perform this merge operation in-place (without using any additional data structures like arrays or lists) and ensure that the final sorted array has a length of $m + n$.

Constraints:

- $0 \le m, n \le 200$
- $1 \le m + n \le 200$
- $-10^9 \le nums1[i], nums2[j] \le 10^9$
- The $nums1$ array has enough space to accommodate all elements from both arrays ($nums1.length == m + n$).

Input Format:

- The first line contains the space-separated integers of the $nums1$ array, where the first m elements represent the scores from the first test, and the remaining n elements are placeholders (0s).
- The second line contains an integer m, representing the number of valid elements in the $nums1$ array.
- The third line contains the space-separated integers of the $nums2$ array, containing the scores from the second test.
- The fourth line contains an integer n, representing the number of elements in the $nums2$ array.

Output Format:

- Output the merged array after merging the two sorted arrays into $nums1$. The merged array should be sorted in non-decreasing order.

Example 1:

Input:

```
1 2 3 0 0 0
3
2 5 6
3
```

Output:

```
[1,2,2,3,5,6]
```

Explanation: The arrays we are merging are [1, 2, 3] and [2, 5, 6].

The result of the merge is [1, 2, 2, 3, 5, 6] with the underlined elements coming from $nums1$.

Program:

CTC27905.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

void merge(int* nums1, int m, int* nums2, int n) {
    int p1 = m - 1;
    int p2 = n - 1;
    int p = m + n - 1;

    while (p2 >= 0) {
        if (p1 >= 0 && nums1[p1] > nums2[p2]) {
            nums1[p] = nums1[p1];
             p1--;
         } else {
            nums1[p] = nums2[p2];
            p2--;
        }
        p--;
    }
}
void printArray(int* arr, int size) {
    printf("[");
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i < size - 1) {
            printf(", ");
        }
    }
    printf("]\n");
}

int main() {
    int   nums1[200];
    int   nums2[100];
    int m, n;
    int temp, i = 0;

    while (scanf("%d", &temp) == 1) {
        nums1[i++] = temp;
        if (getchar() == '\n') {
            break;
        }
    }

    scanf("%d",  &m);

    i = 0;
    while (scanf("%d", &temp) == 1) {
        nums2[i++] = temp;
        if (getchar() == '\n') {
            break;
        }
    }

    scanf("%d", &n);

    merge(nums1, m, nums2, n);
```

CodeTantra

```
    printArray(nums1, m + n);

    return 0;
}
```

Output:

Test case - 1
User Output
1 2 3 0 0 0
3
2 5 6
3
[1, 2, 2, 3, 5, 6]

Test case - 2
User Output
1
1
0
0
[1]

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given a list of N array elements, apply quick sort on the array to arrange the elements in ascending order.

Input Format:

- The first line contains the integer N, the size of the array.
- The next line contains N space-separated integers.

Output Format:

- Print the sorted array as a row of space-separated integers.

Constraints:

- $1 <= N <= 1000$
- $-100000 <= a[i] <= 100000$

Note: Use the divide and conquer approach to solve the question.

Program:

```
CTC14074.c
```

CodeTantra

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
         if (i < n - 1) {
             printf(" ");
        }
    }
    printf(" \n");
    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output

5
4 8 2 -3 0
-3 0 2 4 8

Test case - 2
User Output
8
-6 -2 -8 -36 0 69 -5 -96
-96 -36 -8 -6 -5 -2 0 69

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

Given n items, each with a given **weight** and **value**, you need to determine the **maximum total value** that can be obtained by placing these items into a knapsack with a capacity of w.

Input Format:

The first line contains two space-separated integers:

- n - the number of items.
- w - the maximum capacity of the knapsack.

The second line contains $2 \times n$ space-separated integers:

- Each pair of integers represents the value and weight of an item, respectively.

Output Format:

- A single floating-point number, representing the maximum total value that can be obtained, is formatted to six decimal places.

Example 1:

Input:

```
3 50
60 10 100 20 120 30
```

Output:

```
240.000000
```

Explanation:

Take the item with a value of 60 and a weight of 10, along with the item valued at 100 with a weight of 20. Since the remaining knapsack capacity is 20, a portion of the third item, which has a value of 120 and a weight of 30, is included.

The fraction taken is 120/30, resulting in a value of (120/30) * 20 = 80

Thus, the total maximum value that can be obtained is:

60+100+80=240.000000

This is the optimal value achievable within the given knapsack capacity.

Example 2:

Input:

```
2 50
60 10 100 20
```

Output:

```
160.000000
```

Explanation:

Take both items completely, without breaking. The total maximum value of the item we can have is 160.000000 from the given capacity of a sack.

Constraints:

- $1 \leq n \leq 10^5$
- $1 \leq w \leq 10^9$
- $1 \leq value_i, weight_i \leq 10^4$

Note:

- Utilize the Fractional Knapsack approach, where you can take fractional parts of each item. This allows you to split items and include only a portion of them in the knapsack to achieve the highest possible total value.

Program:

```
CTC31099.c
```

CodeTantra

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int value;
    int weight;
    double
    ratio;
} Item;
int compareItems(const void *a, const void *b) {
    double ratioA = ((Item*)a)->ratio;
    double ratioB = ((Item*)b)->ratio;
    if (ratioA < ratioB) return 1;
    if (ratioA > ratioB) return -1;
    return 0;
}

int main() {
    int n, capacity;
    scanf("%d %d", &n, &capacity);

    Item items[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (double)items[i].value / items[i].weight;
    }

      qsort(items, n, sizeof(Item), compareItems);

    double totalValue =
    0.0; int currentWeight =
    0;
    for (int i = 0; i < n; i++) {
        if (currentWeight + items[i].weight <= capacity) {
            totalValue += items[i].value;
            currentWeight += items[i].weight;
        } else {
            int remainingCapacity = capacity - currentWeight;
            totalValue += items[i].ratio * remainingCapacity;
            currentWeight = capacity;
            break;
        }
    }

    printf("%.6f\n", totalValue);

    return 0;
}
```

CodeTantra

Output:

Test case - 1
User Output
3 50
60 10 100 20 120 30

240.000000

Test case - 2
User Output
2 50
60 10 100 20
160.000000

Result:
Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

You are given a list of N activities, each with a start and end day. The start and end days for each activity are provided in two separate arrays: $start[]$ and $end[]$. Your task is to determine the maximum number of activities that can be completed by a single person. Please note the following:

- A person can only work on one activity per day.
- The duration of an activity includes both the start and end day.
- An activity can only be selected if it does not overlap with the previously selected activity.

Input Format:

- The first line contains an integer N representing the number of activities.
- The second line contains N space separated integers representing the start days.
- The third line contains N space separated integers representing the end days.

Constraints:

- $1 \leq N \leq 10^5$
- $1 \leq start[i] \leq end[i] \leq 10^9$

Output Format:

- Print the total number of activities that a person can do.

Example 1:

Input:

```
2 // N
2 1 // start[] = {2, 1}
2 2 // end[] = {2, 2}
```

Output:

```
1
```

Explanation: Given the overlapping activity durations, a person can only complete one activity.

Example 2:

Input:

```
4
1 3 2 5
2 4 3 6
```

Output:

```
3
```

Explanation: In this case, the person can complete activities 1, 2, and 4 because their time durations do not overlap.

- Activity 1: from 1 to 2
- Activity 2: from 3 to 4
- Activity 4: from 5 to 6

Each activity starts only after the previous one ends, so the person can attend all three without any conflict.

Program:

CTC27805.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int start;
    int end;
} Activity;

int compareActivities(const void *a, const void *b) {
    return ((Activity*)a)->end - ((Activity*)b)->end;
}

int main() {
    int n;
    scanf("%d", &n);

    int start[n];
    int end[n];
    Activity activities[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &start[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &end[i]);
    }

    for (int i = 0; i < n; i++) {
        activities[i].start = start[i];
        activities[i].end = end[i];
    }

    qsort(activities, n, sizeof(Activity), compareActivities);

    int count = 0;
    int lastEndTime = 0;

    for (int i = 0; i < n; i++) {
        if (activities[i].start > lastEndTime) {
            count++;
            lastEndTime = activities[i].end;
        }
    }

    printf("%d\n", count);

    return 0;
}
```

Output:

Test case - 1
User Output
2

CodeTantra

2 1
2 2
1

Test case - 2
User Output
4
1 3 2 5
2 4 3 6
3

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

Aim:

In an automobile company Franky as a manager havenjobs andmworkers. He has given three arrays:difficulty,profit, andworkerwhere:

- difficulty[i]andprofit[i]are the difficulty and the profit of thei^{th}job, and
- worker[j]is the ability ofj^{th}worker (i.e., thej^{th}worker can only complete a job with difficulty at mostworker[j]).

Every worker can be assigned**at most one job**, but one job can be**completed multiple times**.

- For example, if three workers attempt the same job that paysRs 1, then the total profit will beRs 3. If a worker cannot complete any job, their profit isRs 0.

Print the maximum profit we can achieve after assigning the workers to the jobs.

Sample test case 1:

Input:

5 4 ---------- > n and m respectively

2 4 6 8 10 ---------- > difficulty[]

10 20 30 40 50 ---------- profit[]

4 5 6 7---------- > worker[]

Output:

100

Editorial: Workers are assigned jobs of difficulty [4,4,6,6] and they get a profit of [20,20,30,30] separately.

Sample test case 2:

Input:

3 3

85 47 57

24 66 99

40 25 25

Output:

0

Constraints:

- n == difficulty.length
- n == profit.length
- m == worker.length
- 1 <= n, m <= 10^4
- 1 <= difficulty[i], profit[i], worker[i] <= 10^5

Program:

CTC14700.c

CodeTantra

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 10005

typedef struct {
    int difficulty;
    int profit;
} Job;

int compareJobs(const void *a, const void *b) {
    return ((Job*)a)->difficulty - ((Job*)b)->difficulty;
}

int compareWorkers(const void *a, const void *b) {
    return *(int*)a - *(int*)b;
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    Job jobs[MAX_SIZE];
    int difficulty[MAX_SIZE];
    int profit[MAX_SIZE];
    int worker[MAX_SIZE];

    for (int i = 0; i < n; i++) {
        scanf("%d", &difficulty[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &profit[i]);
    }
    for (int i = 0; i < m; i++) {
        scanf("%d", &worker[i]);
    }

    for (int i = 0; i < n; i++) {
        jobs[i].difficulty = difficulty[i];
        jobs[i].profit = profit[i];
    }

    qsort(jobs, n, sizeof(Job), compareJobs);
    qsort(worker, m, sizeof(int), compareWorkers);

    int maxProfit = 0;
    int bestProfit = 0;
    int jobIndex = 0;

    for (int i = 0; i < m; i++) {
        while (jobIndex < n && jobs[jobIndex].difficulty <= worker[i]) {
            if (jobs[jobIndex].profit > bestProfit) {
                bestProfit = jobs[jobIndex].profit;
            }
            jobIndex++;
        }
        maxProfit += bestProfit;
```

CodeTantra

```
    }

    printf("%d\n",  maxProfit);

    return  0;
}
```

Output:

Test case - 1
User Output
5 4
2 4 6 8 10
10 20 30 40 50
4 5 6 7
100

Test case - 2
User Output
3 3
85 47 57
24 66 99
40 25 25
0

Result:

Thus the above program is executed successfully and the output has been verified

CodeTantra

CodeTantra